

Hi,

It's time to enhance stateful processing and provide real-time business insight to our users!

Exercise 1 - stateless into stateful conversion

A year ago you implemented a stateless data ingestion pipeline. It filters out irrelevant events and maps them into the format expected by the sink table. Once formatted, all events are written to the output database and queried in batch to build user navigation sessions. However, this approach has several drawbacks, such as high latency and costs due to the database size needed to handle all ingested data.

Your goal is to transform the stateless pipeline into a stateful pipeline that will create user navigation sessions in real-time and write all finished sessions to an Apache Kafka topic instead of the database. The business rules are:

- Session lifecycle
 - creation
 - only when the job gets an event for a user without an active session
 - expiration
 - each session has 10 minutes TTL
 - a session expires with the watermark
- Session output
 - sessionId: a unique and idempotent string
 - userId: the id of the user associated to the session
 - startTime: the beginning of the session, corresponds to the eventTime of the first visit
 - endTime: the beginning of the session, corresponds to the eventTime of the last visit
 - navigation: a list of visit structure:
 - visit: a structure composed of the page and timeSpent attributes
 - timeSpent is a difference between the event time of the visit and the event time of the next visit expressed in milliseconds
 - timeSpent is null for the last visit
 - all the visits should be ordered by eventTime attribute
 - technicalContext: the values coming from the data enrichment dataset available from /tmp/bde/module6/homework/e1/enrichment_dataset
 - to build the structure, you should join the browserKey and browserVersion from your input dataset with the key and version attributes of the enrichment dataset

Example

Expected output:

```
{"sessionId": "ABC", "userId": 1, "startTime": "2023-06-01T10:00:00.000Z", "endTime":  
"2023-06-01T10:02:10.000Z", "navigation": [  
  {"page": "page1.html", "timeSpent": 30}, {"page": "page2.html", "timeSpent": 100}, {"page":  
"page3.html", "timeSpent": null}  
], "technicalContext": {"browser": {"name": "Firefox", "version": "61.0"}}
```

Input:

1. {"userId": 1, "eventId": "550e8400-e29b-41d4-a716-446655440000", "eventTime":
"2023-06-01T10:00:00.000Z", "page": "page1.html", "browserKey": "ff", "browserVersion":
"61.0"}
2. {"userId": 1, "eventId": "123e8400-e29b-41d4-a716-446655440000", "eventTime":
"2023-06-01T10:00:30.000Z", "page": "page2.html", "browserKey": "ff", "browserVersion":
"61.0"}
3. {"userId": 1, "eventId": "567e8400-e29b-41d4-a716-446655440000", "eventTime":
"2023-06-01T10:02:10.000Z", "page": "page3.html", "browserKey": "ff", "browserVersion":
"61.0"}

The Kafka connection parameters for Apache Spark:

- input
 - kafka.bootstrap.servers: localhost:29092
 - subscribe: visits
 - startingOffsets: earliest
- output
 - kafka.bootstrap.servers: localhost:29092
 - topic: visit_sessions

Setup instructions

Java/Scala

1. Open

https://github.com/bartosz25/master-stream-processing-homework/tree/master/04-stateful-stream-processing/java-scala/e1_and_e3 in IntelliJ.

2. Implement your job. You can use the `com.becomedataengineer.exercise1.StatelessPipeline` as your starting point.

3. Go to `cd docker/e1` and start the containers (`docker-compose down --volumes;`
`docker-compose up`).

4. Go to `com.becomedataengineer.exercise1.DataGenerator` and run it.

5. The `DataGenerator` should continuously write random visits to the *visits* topic.

Python

1. Open

https://github.com/bartosz25/master-stream-processing-homework/tree/master/04-stateful-stream-processing/python/e1_and_e3 in PyCharm.

2. Implement your job. You can use the `e1/stateless_pipeline.py` as your starting point.

3. Go to `cd docker/e1` and start the containers (`docker-compose down --volumes;`
`docker-compose up`).

4. Go to `e1/data_generator.py` and run it.

5. The `data_generator` should continuously write random visits to the *visits* topic.

Exercise 2 - partial results trigger

Your current streaming pipeline uses Apache Flink windows to count the number of visits. However, the results are emitted in the end of each window which doesn't let the business users to follow the traffic real-time. Your goal is to transform the static tumbling window trigger into a dynamic trigger that for a 5-minutes window will emit the results:

- whenever the watermark gets updated during the last 3 minutes

The Kafka connection parameters for Apache Kafka:

- input data:
 - `kafka.bootstrap.servers`: `localhost:29092`
 - `subscribe`: `visits`
 - `startingOffsets`: `earliest`
- output data:
 - `kafka.bootstrap.servers`: `localhost:29092`
 - `topic`: `visits_count`

Setup instructions

Java/Scala

1. Open

<https://github.com/bartosz25/master-stream-processing-homework/tree/master/04-stateful-stream-processing/java-scala/e2> in IntelliJ.

2. Create the partial results trigger.

3. Adapt the `com.becomedataengineer.StatefulFlinkConsumer` job to use the trigger.

3. Go to `cd docker/e2` and start the containers (`docker-compose down --volumes;`
`docker-compose up`).

4. Generate some data by running `com.becomedataengineer.DataGenerator`.

5. The `DataGenerator` should write input data to the *visits* topic.

6. Start a console consumer for the target topic: `docker exec -ti e2_kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic visits_count --from-beginning` . You should see the partial results [here](#).

Python

1. Open

<https://github.com/bartosz25/master-stream-processing-homework/tree/master/04-stateful-stream-processing/python/e2> in PyCharm.

2. Create the partial results trigger.

3. Adapt the `e2/stateful_flink_consumer.py` job to use the trigger.

3. Go to `cd docker/e2` and start the containers (`docker-compose down --volumes;` `docker-compose up`).

4. Generate some data by running `e2/data_generator.py`.

5. The `data_generator` should write input data to the `visits` topic.

6. Start a console consumer for the target topic: `docker exec -ti e2_kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic visits_count --from-beginning` . You should see the partial results [here](#).

Exercise 3 - window-based counters

Users visit your website from one of the 3 browsers: Firefox, Chrome, or Safari. You were asked to write a stateful job that will count the number of visits for each browser in 5-minutes tumbling windows. Additionally, the job should estimate the window completeness.

The expected output should be:

- browser
- window - the window range with start and end attributes
- count - the number of rows in the window so far
- completeness - in milliseconds, how far the window is from the completeness. It's defined as `window.end - max of eventTime seen so far`

For example:

```
{"browser": "Firefox", "window": {"start": "2023-07-14 08:15:00.000Z", "end": "2023-07-14 08:20:00.000Z"}, "count": 30, "completeness": 3}
```

The Kafka connection parameters for Apache Kafka:

- input data:
 - `kafka.bootstrap.servers`: `localhost:29092`
 - `subscribe`: `visits`
 - `startingOffsets`: `earliest`

- output data:
 - kafka.bootstrap.servers: localhost:29092
 - topic: browser_stats

Setup instructions

Java/Scala

1. Open

https://github.com/bartosz25/master-stream-processing-homework/tree/master/04-stateful-stream-processing/java-scala/e1_and_e3 in IntelliJ.

2. Implement your job.

3. Go to `cd docker/e3` and start the containers (`docker-compose down --volumes;`
`docker-compose up`).

4. Go to `com.becomedataengineer.exercise3.DataGenerator` and run it.

5. The `DataGenerator` should continuously write random visits to the *visits* topic.

6. Start a console consumer for the target topic: `docker exec -ti e3_kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic browser_stats --from-beginning`. You should see the results [here](#).

Python

1. Open

https://github.com/bartosz25/master-stream-processing-homework/tree/master/04-stateful-stream-processing/python/e1_and_e3 in PyCharm.

2. Implement your job.

3. Go to `cd docker/e3` and start the containers (`docker-compose down --volumes;`
`docker-compose up`).

4. Go to `e3/data_generator.py` and run it.

5. The `data_generator` should continuously write random visits to the *visits* topic.

6. Start a console consumer for the target topic: `docker exec -ti e3_kafka kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic browser_stats --from-beginning`. You should see the results [here](#).

 **If you are stuck**, please check the *Homework - instructions* document present in the same page. It contains some extra details on how to implement various parts of the homework.

Have fun and see you in the next module!

Bartosz.